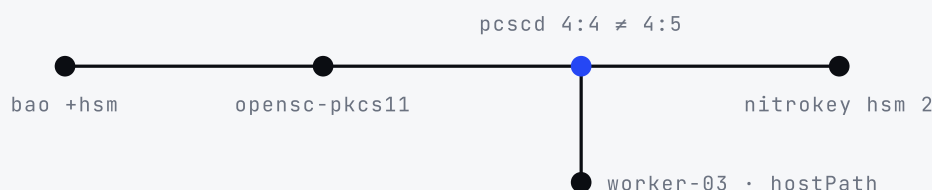


AUTO- UNSEAL OPENBAO

seal "pkcs11" with a USB HSM on the worker: a custom image, the migration from Shamir – and six errors, each of which hid the next.



Auto-Unseal with a Nitrokey HSM 2

seal “pkcs11” for the cluster OpenBao: the USB stick on the worker, a custom image, the migration from Shamir – and six errors, each of which hid the next

Thomas Zachmann

21 September 2026

Contents

What this is about

- How this guide came about

- License and liability

- A word about the values in this guide

- Who this is for

- The building blocks

- The architecture in one picture

Part I – Seal, HSM, PKCS#11: the terms

- What a seal is

- What an HSM adds

- PKCS#11 – the language

- DKEK – the question to ask before the first key

Part II – The stick: Nitrokey HSM 2 on the worker

- USB passthrough in Proxmox

- Packages, pcscd, Polkit

- Initialising the token – with DKEK

- What the token can do – and what not

- Generating the seal key

- The first probe from a pod

Part III – The image

- Why a custom image

- The Dockerfile

- Building and pushing multi-arch

- Trivy

- The protocol problem

- The label problem

Part IV – The chart

- The values

- The Secret with the PIN

- The trap with auditStorage

- The debug pod

Part V – The migration

- Before: snapshot

- Deploy and restart

- `unseal -migrate`

- The proof

- The way back (not carried out)

Part VI – What went wrong, and why

Part VII – Operations and failure behaviour

What can fail now

Recovery keys: what they can and cannot do

They are the old keys – and why to rotate them now

The key is not backed up

Snapshots remain mandatory

Checking that it runs

Part VIII – Debug: the network HSM does not answer (procedure)

The chain

Step 1 – Can you get there at all?

Step 2 – Is the library there, and does it load?

Step 3 – Does the library see a slot?

Step 4 – Can the token do what OpenBao needs?

Step 5 – Login and key

Step 6 – One operation, if possible

Step 7 – What the HSM itself says

Step 8 – Hold the OpenBao configuration against what you found

Step 9 – Mounts and Secrets

Step 10 – Now the OpenBao logs

Step 11 – From the node yes, from the pod no

Step 12 – When everything is right and it still does not work: strace

The order, as a table

Part IX – One HSM for all pods: the HSM VM (draft)

Way A – Transit seal via the VM OpenBao (recommended)

Way B – p11-kit remote: the PKCS#11 library over the network

Way C – USB/IP

Way D – Nitrokey NetHSM

Recommendation

Part X – OpenBao 2.7: from HSM build to KMS plugin (draft)

The stumbling blocks already visible

Appendix A – All commands

Appendix B – Glossary

About the author

What this is about

N° 1 ended with a ritual: after every pod restart, somebody types three of five unseal keys into `bao operator unseal`. Until then OpenBao is there, but dead – 0/1 READY, no secret comes out, every application waiting for it keeps waiting. A node update at three in the morning means somebody gets up at three in the morning.

This guide abolishes the ritual. The root key, which the Shamir keys used to reconstruct, is afterwards encrypted with a key that never leaves a **Nitrokey HSM 2** – a USB stick plugged into the cluster worker. OpenBao starts, asks the HSM, is unsealed. No human, no file with keys, no cloud KMS.

The way there was longer than planned. The prepared plan (N° 6 in its first draft) called for an AES key on the HSM and the stick on the VM from N° 3. Neither held: the Nitrokey HSM 2 cannot do AES-GCM, and the stick ended up on the Kubernetes worker. The official OpenBao image cannot do PKCS#11. The PKCS#11 library in the custom image spoke a protocol the daemon on the host rejects. The token had a different name than the test tool claimed. And an empty Secret looked like a full one. Six errors, each trivial on its own, together an afternoon – and the actual content of this guide.

It is the sixth part of a series. It assumes the cluster OpenBao from N° 1 and the snapshot from N° 7, without which one should not start a seal migration. Three parts at the end are not a session but procedures and drafts, and marked as such: a debug procedure for an HSM reached over the network; an HSM that does not hang off one worker but serves all pods; and the path OpenBao 2.7 prescribes once it discontinues the HSM distribution.

How this guide came about

The other notes in this series were written without AI assistance. This one was not: the rebuild on the cluster ran as a pair-programming session with Claude Code, which contributed research, debugging pods and the first draft of the text. Every command was executed on the cluster, every output is real, every decision was mine. The yardstick stays the same: whoever has worked through this guide can draw, on a blank sheet of paper, the chain from the bao process to the chip in the stick and say at which four points it can break.

License and liability

This guide is published under CC BY 4.0: it may be copied, shared and adapted, commercially too, as long as the author is credited. It is provided without warranty. Everything in it was carried out in a development environment; whoever reproduces it elsewhere does so at their own risk. Running an HSM in production takes more than what is written here – Part VII says what.

A word about the values in this guide

The HSM's user PIN, the SO PIN, the unseal keys and the root token appear nowhere – not even as development values. The PIN went into a Kubernetes Secret as a file, and the file into `rm -P` afterwards. Internal addresses, the external hostname and the stick's serial number are replaced by

placeholders (<worker-03-ip>, openbao.example.internal, DENK0000000). The image softxpert/openbao-pkcs11 is public and real.

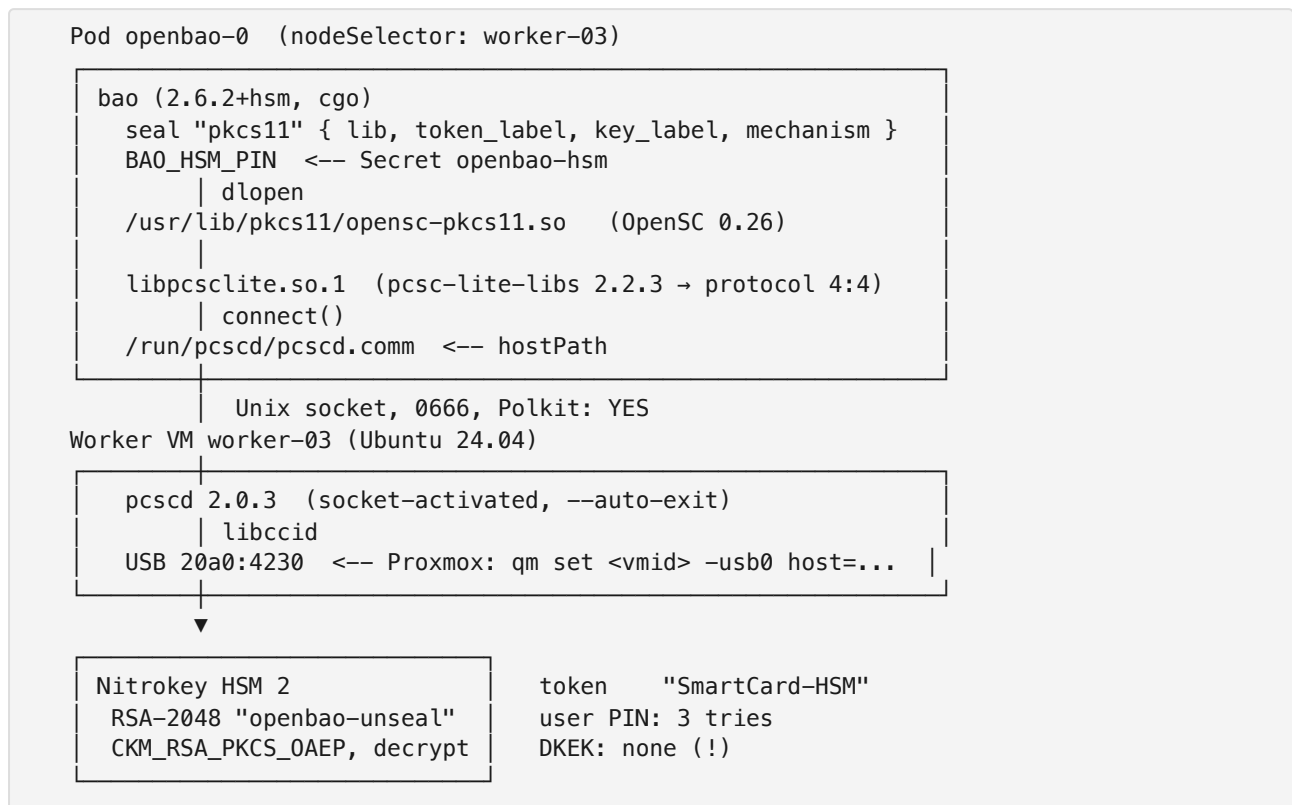
Who this is for

For readers who know N° 1 and have never held an HSM. Part I explains what a seal is, what an HSM adds to it, and what the terms PKCS#11, token, slot, SO PIN and DKEK mean – whoever knows that skips it. OpenBao fundamentals are in my book **Vault in Practice** (Leanpub, leanpub.com/vault-in-practice).

The building blocks

Building block	Version	Role
OpenBao in the cluster (N° 1)	2.6.2, chart 0.29.4, standalone, Raft	the thing to be unsealed
Nitrokey HSM 2	firmware 4.1 (SmartCard-HSM)	holds the seal key openbao-unseal, RSA 2048
Proxmox	–	passes the USB stick to the worker VM via <code>qm set -usb0</code>
Worker VM worker-03	Ubuntu 24.04, pcscd 2.0.3, libccid, OpenSC 0.25	talks to the stick; pcscd listens on <code>/run/pcscd/pcscd.comm</code>
Image softxpert/openbao-pkcs11:2.6.2	openbao/openbao-hsm:2.6.2 + OpenSC 0.26 + pcsc-lite-libs 2.2.3	OpenBao with PKCS#11 support and the client library
Helm values	<code>server.image</code> , <code>nodeSelector</code> , <code>hostPath /run/pcscd</code> , <code>seal "pkcs11"</code> , <code>extraSecretEnvironmentVars</code>	ties it all together
Secret openbao-hsm	BA0_HSM_PIN, 8 bytes	the user PIN, only in the cluster

The architecture in one picture



Three things to take away from the picture:

1. **The chain has four interfaces, and each has its own failure mode.** bao ↔ library (build without PKCS#11), library ↔ daemon (protocol version), daemon ↔ reader (Polkit, CCID), reader ↔ chip (PIN, label, key). Part VI has an entry for each.
2. **The pod is bound to the worker.** A USB stick follows no pod. If worker-03 dies there is no unseal – not with recovery keys, not with anything. That is a deliberate single point of failure, not an oversight, and Part IX shows how to get rid of it.
3. **The key is not backed up.** The token was initialised without a DKEK. The seal key exists exactly once, in the chip. Part VII says what that means and what has to happen before production use.

Part I – Seal, HSM, PKCS#11: the terms

What a seal is

OpenBao encrypts everything it stores with a **root key** (older texts say master key). The root key itself lies in storage – encrypted. With what, that is what the **seal** decides:

- **Shamir** (N° 1): the root key is split into five parts using Shamir's Secret Sharing, three suffice to reconstruct it. The parts are the unseal keys. After every start, three of them must be entered; only then can OpenBao form the root key and open the barrier.
- **Auto seal**: the root key is encrypted with a key that lives outside OpenBao – in a cloud KMS, in a second OpenBao (transit) or in an HSM. OpenBao asks that system at start, gets the root key back decrypted and opens itself.

With an auto seal the five keys remain, but are called **recovery keys** and do something else: they authorise a root token reset and the way back to Shamir. They **cannot** reconstruct the root key. Whoever loses their HSM has, with recovery keys, a safe without a key.

What an HSM adds

A **hardware security module** is a device that generates, stores and uses cryptographic keys without ever handing them out. You send it data, it answers with the result – signed, decrypted, wrapped. The private key does not leave the chip; making a copy is not provided for, and reading it out is meant to fail physically too.

The Nitrokey HSM 2 is the small form of that: a USB stick with a SmartCard-HSM card from CardContact. It does what a rack HSM does, slowly and for one set of keys. For a seal key that is enough – OpenBao needs the HSM only at start and every ten minutes for a health check.

PKCS#11 – the language

PKCS#11 (also “Cryptoki”) is the standard API through which programs talk to HSMs and smartcards. It is a C library – a .so file – that a program loads via dlopen. The terms in it:

- **Slot** – a socket, physical or logical. Here: the USB reader.
- **Token** – the card in the slot. It has a **label** (a name), a serial number and flags (“login required”, “PIN initialized”).
- **Objects** – what lies on the token: keys, certificates, data. Every object has a label and an ID. Private keys are only visible after login.
- **Mechanisms** – what the token can compute: CKM_RSA_PKCS_OAEP, CKM_AES_GCM, ... Every token can do a different selection. What it cannot do, OpenBao cannot do with it.
- **User PIN** – the login for key operations. Three failed attempts, then it is locked.
- **SO PIN** (security officer) – the administrative login: initialise the token, reset the user PIN, wipe the token. 15 attempts. OpenBao never gets to see it.

For the Nitrokey HSM 2, **OpenSC** provides the PKCS#11 library (`opensc-pkcs11.so`) and the tools (`pkcs11-tool`, `sc-hsm-tool`, `opensc-tool`). OpenSC does not talk to USB itself but to the **PC/SC daemon** `pcscd` over a Unix socket; `pcscd` talks to the reader via the **CCID** driver. Four layers, all of them have to fit.

DKEK – the question to ask before the first key

An HSM does not hand out keys. What if it breaks? The SmartCard-HSM has the **Device Key Encryption Key** for that: if the token is initialised with one or more DKEK shares, keys can later be exported encrypted with the DKEK (`sc-hsm-tool --wrap-key`) and imported onto a second stick initialised with the same shares. Without DKEK: no export, no replacement stick, no backup. The decision is made at `--initialize` and can only be changed afterwards by re-initialising – which wipes all keys.

The token in this guide was initialised **without** DKEK. What that means is in Part VII. Whoever rebuilds this should do it differently, and Part II shows how.

Part II – The stick: Nitrokey HSM 2 on the worker

USB passthrough in Proxmox

The workers are VMs. The stick sits in the Proxmox host and has to be passed to the VM – once, on the host, before anything happens on the VM:

```
# on the Proxmox host
lsusb | grep -i nitrokey
#   Bus 001 Device 005: ID 20a0:4230 Clay Logic Nitrokey HSM

# bind it to the VM – by vendor:product, not by bus/port: survives
# re-plugging into another port
qm set <vmid> -usb0 host=20a0:4230

# a cold boot is the reliable way; hot-plug works on recent QEMU but is
# not worth the uncertainty for an HSM
qm shutdown <vmid> && qm start <vmid>
```

In the VM:

```
$ lsusb | grep -i nitrokey
Bus 002 Device 002: ID 20a0:4230 Clay Logic Nitrokey HSM
```

Packages, pcscd, Polkit

```
sudo apt install -y pcscd libccid pcsc-tools opensc
sudo systemctl enable --now pcscd
```

Ubuntu 24.04 ships pcscd 2.0.3, libccid 1.5.5, OpenSC 0.25. pcscd is **socket-activated**: pcscd.socket listens on /run/pcscd/pcscd.comm, the service starts with the first client and exits again after idling (--auto-exit). systemctl is-active pcscd therefore often shows inactive while pcscd.socket is active – that is not an error.

```
$ pcsc_scan
Reader 0: Nitrokey Nitrokey HSM (DENK00000000 ) 00 00
Card state: Card inserted,
ATR: 3B DE 18 FF 81 91 FE 1F C3 80 31 81 54 48 53 4D 31 73 80 21 40 81 07 FA
```

Whoever is not root sees nothing at first: pcscd asks **Polkit** whether the requesting process may use the reader, and the default says no to everything that does not sit in a local session. A container process sits in none. The rule that opens it:

```
// /etc/polkit-1/rules.d/40-allow-pcsd.rules
polkit.addRule(function(action, subject) {
    if (
        action.id == "org.debian.pcsd-lite.access_pcsd" ||
        action.id == "org.debian.pcsd-lite.access_card"
    ) {
        return polkit.Result.YES;
    }
});
```

```
sudo systemctl restart polkit pcsd
```

That is deliberately coarse: every process on the host may reach the reader. Justifiable on a worker that only runs pods anyway; on a multi-user machine one would check `subject.user`. The socket itself is `0666` (`srw-rw-rw-`); the actual access control is Polkit.

Initialising the token – with DKEK

A factory-fresh stick has a transport SO PIN (3537363231383830) and has to be initialised. Three decisions are made in the process: SO PIN, user PIN, DKEK shares. **This is the step that went differently in this guide than it should have** – the token was initialised without DKEK shares. This is what it would look like done right:

```
# SO PIN: 16 hex characters. User PIN: 6–15 characters. Both are prompted,
# not passed as arguments (shell history!).
sc-hsm-tool --initialize --dkek-shares 1 --label openbao

# create and import the DKEK share – the share file is the HSM's backup.
# It is password-protected; keep file and password apart.
sc-hsm-tool --create-dkek-share dkek-share-1.pbe
sc-hsm-tool --import-dkek-share dkek-share-1.pbe

sc-hsm-tool                                # must show "DKEK shares : 1"
```

With a DKEK, every key generated later can be exported as an encrypted file (`--wrap-key`) and imported onto a replacement stick initialised with the same share file. Without DKEK there is no replacement stick.

The state here:

```
$ sc-hsm-tool
Using reader with a card: Nitrokey Nitrokey HSM (DENK00000000 ) 00 00
Version                : 4.1
Config options         :
    User PIN reset with SO-PIN enabled
SO-PIN tries left      : 15
User PIN tries left    : 3
```

No line DKEK shares. What lies on this token lies only there.

What the token can do – and what not

Before generating a key, ask which mechanisms the token supports. For the seal, OpenBao needs an AEAD scheme: CKM_AES_GCM or CKM_RSA_PKCS_OAEP.

```
$ pkcs11-tool -M | grep -E 'AES|OAEP'
RSA-PKCS-OAEP, keySize={1024,4096}, hw, decrypt
```

No AES-GCM. The prepared plan – AES-256, mechanism = "0x1087" – would have failed at the token. RSA-OAEP is there, with decrypt in hardware. This is how the seal works then: OpenBao generates a random AES key, encrypts the root key with it, wraps the AES key with the public RSA key (in software) and stores both. On unseal it sends the wrapped AES key to the HSM, which decrypts it with the private RSA key. The private key stays in the chip.

Generating the seal key

An RSA-2048 pair, label openbao-unseal, ID 10, usage decrypt:

```
pkcs11-tool --login --keypairgen --key-type rsa:2048 \
--label openbao-unseal --id 10 --usage-decrypt
```

The user PIN is prompted. Afterwards – without login one only sees the public part, that is intended:

```
$ pkcs11-tool -0
Using slot 0 with a present token (0x0)
Public Key Object; RSA 2048 bits
label:      openbao-unseal
ID:         10
Usage:      encrypt, verify, wrap
```

(The key in this guide was generated before the recorded session; the command above is the standard procedure that leads to exactly this object.)

The first probe from a pod

Before OpenBao enters the picture: does a container see the stick? A throwaway pod on worker-03 with the pcscd socket as hostPath:

```

apiVersion: v1
kind: Pod
metadata: {name: pkcs11-test, namespace: openbao}
spec:
  nodeSelector: {kubernetes.io/hostname: worker-03}
  securityContext: {runAsNonRoot: true, runAsUser: 1000, runAsGroup: 1000}
  containers:
  - name: test
    image: debian:bookworm-slim          # later: custom image with opensc
    command: ["sleep", "infinity"]
    volumeMounts: [{name: pcscd, mountPath: /run/pcscd}]
  volumes:
  - name: pcscd
    hostPath: {path: /run/pcscd, type: Directory}

```

```

$ kubectl exec -n openbao pkcs11-test -- pkcs11-tool -L
Available slots:
Slot 0 (0x0): Nitrokey Nitrokey HSM (DENK00000000 ) 00 00
  token label      : SmartCard-HSM (UserPIN)
  token manufacturer : www.CardContact.de
  token model      : PKCS#15 emulated
  token flags      : login required, rng, token initialized, PIN initialized

```

Works – as UID 1000, from Debian bookworm, with OpenSC 0.23. This result later produced two false certainties: that the chain pod → socket → pcscd works in principle (true), and that the token is called SmartCard-HSM (UserPIN) (true only for this OpenSC version).

Part III – The image

Why a custom image

The official `openbao/openbao` contains no OpenSC and – more importantly – a `bao` binary compiled without PKCS#11 support. The seal needs CGO and the build tag `hsm`. OpenBao ships a second distribution for that: `openbao/openbao-hsm` (Alpine) and `openbao/openbao-hsm-ubi` (RHEL UBI, glibc). Both on Docker Hub and Quay, amd64 and arm64.

The first attempt took the wrong base. The Dockerfile was formally correct, the image built, the pod started – and died:

```
Error configuring seal "pkcs11": this build of OpenBao has PKCS#11 disabled
```

The libraries were there. The program that was supposed to load them could not. The message is unambiguous once you know there are two builds.

The Dockerfile

```
FROM openbao/openbao-hsm:2.6.2

USER root

# pcsc-lite-libs is pinned to 2.2.x from Alpine 3.21 on purpose: pcsc-lite >= 2.3
# speaks client protocol 4:5, but pcsd 2.0.3 on the Ubuntu 24.04 worker only
# accepts 4:4 and rejects newer clients ("Communication protocol mismatch").
# Revisit when the worker's pcsd is upgraded or the base image changes.
RUN apk add --no-cache \
    --repository https://dl-cdn.alpinelinux.org/alpine/v3.21/main \
    opensc \
    'pcsc-lite-libs=2.2.3-r1'

USER openbao
```

Three lines do something, and each has a story:

- `FROM openbao/openbao-hsm:2.6.2` – the build with PKCS#11. `bao` version says OpenBao v2.6.2+hsm ... (`cgo`). Alpine 3.24, user `openbao` with UID 100 – the same UID as in the standard image, nothing in the chart has to change.
- `opensc` – the PKCS#11 library `/usr/lib/pkcs11/opensc-pkcs11.so` and the tools. Alpine does not pull in `pcsc-lite-libs` automatically; without the package OpenSC reports “Unable to load external module”.
- `pcsc-lite-libs=2.2.3-r1` from the **3.21** repository instead of 2.4.0 from 3.24 – the second error of this guide, explanation follows.

Building and pushing multi-arch

The cluster is amd64, the Mac arm64. `docker build` yields only the host architecture; for both it takes BuildKit:

```
docker buildx build --platform linux/amd64,linux/arm64 \
  -t softxpert/openbao-pkcs11:2.6.2 --push .
```

`--push` is not a convenience here: a multi-arch image is a manifest list pointing to one image per platform. The classic local image store cannot hold such a thing; BuildKit exports straight to the registry. The amd64 part is built on the Mac via QEMU emulation – uncritical for `apk add`.

```
$ docker buildx imagetools inspect softxpert/openbao-pkcs11:2.6.2
Digest:      sha256:265babb4f237b03cbcd94abaaf84a9cb5f75d74acecf88a867685f792564e2c5
Platform:    linux/amd64
Platform:    linux/arm64
```

The digest is the reference that belongs in the values – not the tag. The tag 2.6.2 was overwritten three times in this session (wrong base, right base, pinned library). With `pullPolicy: IfNotPresent` the worker would have kept the cached old copy every time.

Trivy

```
$ trivy image --platform linux/amd64 --severity HIGH,CRITICAL softxpert/openbao-pkcs11:2.6.2
softxpert/openbao-pkcs11:2.6.2 (alpine 3.24.1)  2 HIGH  (libcrypto3/libssl3, OpenSSL QUI
C DoS, fixed in 3.5.8-r0)
usr/bin/bao (gobinary)                          9 HIGH
```

All eleven findings come from the base image; `opensc` and `pcsc-lite-libs` added not a single one. Five of the nine in the Go binary are false positives: Trivy reports OpenBao CVEs that, by its own table, are fixed in 2.0.3 to 2.5.4 – but the binary carries a Go pseudo-version (`v0.0.0-20260818...`) instead of `v2.6.2`, and Trivy cannot compare it. The two OpenSSL findings could be fixed with `apk upgrade --no-cache`.

The protocol problem

The image with the right base started, found the PIN – and then:

```
Error configuring seal "pkcs11": failed to find token with label: SmartCard-HSM (UserPIN)
```

The container lived too briefly for a `kubectl exec`. So a debug pod with **the same image, the same UID, the same mounts, the same node**, running only `sleep`:

```
$ kubectl exec -n openbao pkcs11-debug -- opensc-tool -l
No smart card readers found.
```

The same socket at which the Debian test pod had seen the stick minutes before. The answer was in the host's journal:

```
$ sudo journalctl -u pcscd --since -15min
pcscd[530642]: winscard_svc.c:402:ContextThread() Communication protocol mismatch!
pcscd[530642]: winscard_svc.c:404:ContextThread() Client protocol is 4:5
pcscd[530642]: winscard_svc.c:406:ContextThread() Server protocol is 4:4
```

pcsc-lite raised the minor version of its client-server protocol with version 2.3.0, and pcscd rejects clients with a different minor version. Empirically, with throwaway pods from different Alpine versions against the same host:

Alpine	pcsc-lite-libs	protocol	against pcscd 2.0.3
3.21	2.2.3	4:4	✓ token visible
3.22	2.3.3	4:5	✗ "No smart card readers"
3.24	2.4.0	4:5	✗
Debian bookworm	1.9.9	4:4	✓ (the test pod)

Ubuntu 24.04 stays on pcscd 2.0.3 – the host cannot be lifted without third-party packages. The client side can be: one line in the Dockerfile. That is a cross-release package and deserves a comment, so that at the next base image change somebody checks whether it is still needed.

The label problem

With the pinned library, from the debug pod:

```
$ kubectl exec -n openbao pkcs11-debug -- pkcs11-tool -L
Slot 0 (0x0): Nitrokey Nitrokey HSM (DENK00000000) 00 00
token label      : SmartCard-HSM
```

Not SmartCard-HSM (UserPIN). The label is not an attribute of the chip but is generated by OpenSC's PKCS#15 emulation – and OpenSC 0.26 generates a different one than 0.23 and 0.25. `token_label` in the seal stanza has to be what **the OpenSC version in the image** reports. Check after every rebuild. The alternative `slot = "0"` is stable with a single reader, but less readable.

Part IV – The chart

The values

The changes to helm/values.yaml from N° 1, everything under server::

```
server:
  # chart default registry is quay.io – without "registry",
  # quay.io/softxpert/openbao-pkcs11 would be pulled: ImagePullBackOff.
  image:
    registry: docker.io
    repository: softxpert/openbao-pkcs11
    tag: "2.6.2@sha256:265babb4f237b03cbcd94abaaf84a9cb5f75d74acecf88a867685f792564e2c5"
    pullPolicy: IfNotPresent

  # the stick hangs off worker-03. Deliberate SPOF (Part IX).
  nodeSelector:
    kubernetes.io/hostname: worker-03

  # mount the directory, not the socket file: pcscd recreates the socket
  # on every start; a file mount would point at the dead inode.
  # no readOnly – connect() on a unix socket needs write permission.
  volumes:
    - name: pcscd
      hostPath:
        path: /run/pcscd
        type: Directory
  volumeMounts:
    - name: pcscd
      mountPath: /run/pcscd

  standalone:
    enabled: true
    config: |
      ui = true
      listener "tcp" { ... }           # as in N° 1
      storage "raft" { ... }          # as in N° 1

      # token_label is what OpenSC *in this image* reports (pkcs11-tool -L):
      # 0.26 says "SmartCard-HSM", older versions "SmartCard-HSM (UserPIN)".
      seal "pkcs11" {
        lib      = "/usr/lib/pkcs11/opensc-pkcs11.so"
        token_label = "SmartCard-HSM"
        key_label  = "openbao-unseal"
        mechanism  = "CKM_RSA_PKCS_OAEP"
      }

  # the PIN comes from the Secret, never from the file
  extraSecretEnvironmentVars:
    - envName: BAO_HSM_PIN
      secretName: openbao-hsm
      secretKey: BAO_HSM_PIN
```

Four points on that:

- **tag with digest.** The chart renders `registry/repository:tag`; a tag of the form `2.6.2@sha256:...` yields a valid reference in which the digest wins. No new chart parameter needed.
- **pin is not in the stanza.** Every seal parameter has an environment variable (`BA0_HSM_PIN`, `BA0_HSM_LIB`, ...), and the variable takes precedence. The ConfigMap with the stanza lies readable in the cluster; the Secret does not.
- **mechanism is explicit**, although OpenBao could derive it from the key type. The repo should state what the HSM does.
- **updateStrategyType: OnDelete** is the chart default. A `helm upgrade` changes the StatefulSet template but replaces no running pod. Every change here only takes effect with `kubectl delete pod openbao-0`.

The Secret with the PIN

```
# in a separate terminal, zsh:
read -rs "PIN?HSM user PIN: "; printf '%s' "$PIN" > /tmp/hsm-pin; unset PIN
wc -c < /tmp/hsm-pin                # 6-15, otherwise something is wrong

kubectl -n openbao create secret generic openbao-hsm \
  --from-file=BA0_HSM_PIN=/tmp/hsm-pin
rm -P /tmp/hsm-pin                # macOS; Linux: shred -u

# and CHECK – kubectl does not validate values:
kubectl -n openbao get secret openbao-hsm -o json \
  | python3 -c "import sys,json,base64; d=json.load(sys.stdin)['data']; print({k: len(base64.b64decode(v)) for k,v in d.items()})"
{'BA0_HSM_PIN': 8}
```

The last line is not decoration. The first attempt of this guide produced a Secret with **0 bytes**: the instructions said `read -rs -p "Prompt" PIN – bash syntax`. In zsh, `read -p` means “read from the coprocess”, `$PIN` stayed empty, `--from-literal=BA0_HSM_PIN=""` created an empty Secret, and OpenBao said `pin` is required. The Secret existed, had the right key, looked like any other in `kubectl get secret`.

The trap with auditStorage

Not part of the seal, but part of this rebuild: the values got an `auditStorage` (its own volume for the audit device). That creates a second `volumeClaimTemplate` in the StatefulSet – and Kubernetes forbids that on an existing StatefulSet:

```
updates to statefulset spec for fields other than 'replicas', 'ordinals',
'template', 'updateStrategy', 'persistentVolumeClaimRetentionPolicy' and
'minReadySeconds' are forbidden
```

`helm --dry-run=server` does **not** see this – it renders with server lookups but does not send the manifests to the API server with `dryRun`. The way: `kubectl delete sts openbao --cascade=orphan` (the pod keeps running), `helm upgrade` creates the StatefulSet anew and adopts the pod via its labels, then `delete pod`. The PVC stays thanks to `persistentVolumeClaimRetentionPolicy: Retain`.

The debug pod

The most important tool of this guide, for the appendix:

```
apiVersion: v1
kind: Pod
metadata: {name: pkcs11-debug, namespace: openbao}
spec:
  nodeSelector: {kubernetes.io/hostname: worker-03}
  restartPolicy: Never
  securityContext: {runAsNonRoot: true, runAsUser: 100, runAsGroup: 1000}
  containers:
  - name: debug
    image: docker.io/softxpert/openbao-pkcs11:2.6.2@sha256:265babb4...
    command: ["sleep", "1800"]
    volumeMounts: [{name: pcscd, mountPath: /run/pcscd}]
  volumes:
  - name: pcscd
    hostPath: {path: /run/pcscd, type: Directory}
```

Same image, same UID 100, same mount, same node – just sleep instead of bao. Inside: `opensc-tool -l, pkcs11-tool -L, -0, -M`. What works here works in OpenBao too; what fails here fails there with a worse error message.

Part V – The migration

Before: snapshot

```
kubectl -n openbao exec -it openbao-0 -- sh -c \  
  'bao login -method=userpass username=snapshot >/dev/null && \  
    bao operator raft snapshot save /tmp/pre-pkcs11.snap'  
kubectl -n openbao cp openbao-0:/tmp/pre-pkcs11.snap ./pre-pkcs11-2026-09-21.snap
```

Non-negotiable, and with a deadline: a snapshot needs an **unsealed** OpenBao. As soon as the pod restarts with the seal stanza it is sealed until the migration is through – the window is closed then. The first run here was aborted for exactly that reason and the snapshot taken afterwards.

Deploy and restart

```
$ helm upgrade openbao openbao/openbao --version 0.29.4 -n openbao -f helm/values.yaml  
Release "openbao" has been upgraded. Happy Helming!  
REVISION: 8  
  
$ kubectl -n openbao delete pod openbao-0  
$ kubectl -n openbao logs openbao-0 | grep -E 'seal|Seal'  
core: entering seal migration mode; Vault will not automatically unseal even  
      if using an autoseal: from_barrier_type=shamir to_barrier_type=pkcs11  
Auto Seal: pkcs11 (builtin: true, key_label: "openbao-unseal",  
             lib: "/usr/lib/pkcs11/opensc-pkcs11.so", mechanism: "CKM_RSA_PKCS_OAEP",  
             token_label: "SmartCard-HSM")  
  
$ kubectl -n openbao exec openbao-0 -- bao status  
Seal Type           pkcs11  
Recovery Seal Type  shamir  
Initialized          true  
Sealed               true  
Total Recovery Shares 5  
Threshold            3  
Seal Migration in Progress true  
Version              2.6.2+hsm
```

OpenBao has found the HSM, recognised the key, accepted the PIN – and knows that the root key in storage is still Shamir-encrypted. It waits.

unseal -migrate

```
kubectl -n openbao exec -it openbao-0 -- bao operator unseal -migrate  
Unseal Key (will be hidden):
```

Three times, one Shamir key each. On the third the work happens:

```
core: migrating from shamir to auto-unseal: to=pkcs11
core: seal migration complete
core: post-unseal setup complete
```

```
$ bao status
Seal Type          pkcs11
Recovery Seal Type shamir
Sealed             false
Total Recovery Shares 5
Threshold          3
```

Seal Migration in Progress is gone. The five keys are now recovery keys. `-migrate` is thereby used up – a second time would be an error (“no migration seal found”, the same message as for a pod that does not have the stanza yet; the first attempt here ran against exactly such a pod).

The proof

Migration successful does not mean auto-unseal works. The test that matters: delete the pod, enter **nothing**.

```
$ kubectl -n openbao delete pod openbao-0
$ kubectl -n openbao logs openbao-0 | grep -E 'unseal'
core: stored unseal keys supported, attempting fetch
core: vault is unsealed
core: unsealed with stored key

$ kubectl -n openbao get pod openbao-0
NAME          READY   STATUS    RESTARTS   AGE
openbao-0     1/1     Running   0           21s

$ curl -s -o /dev/null -w '%{http_code}\n' https://openbao.example.internal/v1/sys/health
200
```

2.4 seconds between attempting `fetch` and `unsealed`. That is the RSA decryption in the stick plus four layers in between.

The way back (not carried out)

Should the HSM be replaced or the seal go back to Shamir: keep the stanza, add `disabled = "true"`, restart the pod, then `bao operator unseal -migrate` – this time with three **recovery** keys. OpenBao decrypts the root key one last time via the HSM and re-encrypts it for Shamir. Prerequisite: the HSM is still there. The way back needs both seals at the same time. That is why a dying HSM has to be acted on immediately – not once it is dead.

This step was not carried out here. It belongs in the restore drill from N° 7, before the seal counts as production.

Part VI – What went wrong, and why

Error	Symptom	Cause	Fix
Wrong base image	this build of OpenBao has PKCS#11 disabled	openbao/openbao is built without CGO/hsm	FROM openbao/openbao-hsm:2.6.2
Empty Secret	pin is required; Secret exists	zsh: read -p reads from the coprocess, \$PIN empty	zsh syntax read -rs "PIN?...", check the length
Protocol mismatch	failed to find token; in the debug pod “No smart card readers”; host journal Client 4:5, Server 4:4	pcsc-lite ≥ 2.3 in the image vs. pcscd 2.0.3 on Ubuntu 24.04	pin pcsc-lite-libs=2.2.3-r1 from Alpine 3.21
Wrong token label	failed to find token with label: SmartCard-HSM (UserPIN)	label comes from OpenSC’s PKCS#15 emulation; 0.26 ≠ 0.23/0.25	read the label from the target image: SmartCard-HSM
No migration seal	can't perform a seal migration, no migration seal found	unseal -migrate against a pod without the stanza (deploy was missing)	order: deploy → pod → -migrate
Immutable STS	helm upgrade would change volumeClaimTemplates	Kubernetes forbids it; --dry-run=server does not check it	delete sts --cascade=orphan, then helm upgrade
Wrong registry prefix (avoided)	would be ImagePullBackOff	chart default registry: quay.io	registry: docker.io explicitly
Stale image (avoided)	would be old image despite push	same tag + IfNotPresent + node cache	digest in tag

And two that had no error message: the prepared plan called for AES-GCM, which the token cannot do – noticed only at `pkcs11-tool -M`. And the token was initialised without DKEK – noticed only while writing this guide, at `sc-hsm-tool`. Both would have been known before the first key if the two commands had been run first.

Part VII – Operations and failure behaviour

What can fail now

Before the rebuild there was one failure mode: pod restarts, somebody types keys. Now there are five, and none is solved by typing:

Fails	Symptom	What helps
worker-03	pod Pending (nodeSelector), Raft PVC stuck	stick into another worker, <code>qm set</code> on the other VM, change nodeSelector, Polkit rule there
USB stick	failed to find token, health-check warnings in the log	replacement stick – only with a DKEK share (see below)
pcscd on the host	same symptom	<code>systemctl restart pcscd.socket</code> ; read the journal
Secret openbao-hsm	pin is required	recreate the Secret, restart the pod
User PIN locked	CKR_PIN_LOCKED	<code>sc-hsm-tool --unlock-pin</code> with the SO PIN

On the last row: the user PIN has **three tries**. A pod in CrashLoop with a wrong PIN in the Secret uses them up in under a minute. After that the token is locked, and only the SO PIN opens it again. Whoever changes the PIN checks the Secret (length check!) before restarting the pod.

OpenBao runs a **seal health check** every ten minutes – encrypts and decrypts a random value via the HSM – and warns in the log when it fails. That is the line a log alert belongs on: it comes hours before the restart that then no longer works.

Recovery keys: what they can and cannot do

- **Can:** generate a new root token (`generate-root`), authorise the way back to Shamir (`unseal – migrate with disabled = "true"`), reopen a manually sealed OpenBao (`bao operator seal`) – but only if the HSM is reachable; they merely authorise then.
- **Cannot:** reconstruct the root key. Without the HSM the data is encrypted, and the recovery keys are paper.

They still belong in the password manager, in the same place as the unseal keys before – and the runbook from N° 1 has to be renamed: after a restart there is nothing to unseal; whoever tries gets an error.

They are the old keys – and why to rotate them now

A misunderstanding I had myself: on a *fresh initialisation* with an HSM seal (`bao operator init` against a new instance), OpenBao generates new recovery keys and prints them once. On a *migration* that does not happen. `unseal -migrate` repurposes the five Shamir keys – the same key material, a new role. `bao status` shows it: Recovery Seal Type shamir, Total Recovery Shares 5, Threshold 3 – the old parameters.

That also means: the keys that were typed into a terminal three times during the migration are the keys that will authorise the way back from now on. Whoever had them on several machines, in a screen share or in a shell history during that session rotates them now – the migration is the natural moment, because the old ones are at hand anyway:

```
kubectl -n openbao exec -it openbao-0 -- bao operator rekey \
  -target=recovery -init -key-shares=5 -key-threshold=3
# output: a nonce. Then three times, one OLD recovery key each:
kubectl -n openbao exec -it openbao-0 -- bao operator rekey -target=recovery -nonce=<nonce
>
```

On the third key OpenBao prints **five new recovery keys** – once, in the terminal. The old ones are invalid from that moment. The new ones go into the password manager before anything else happens (N° 7, the error with the shell variables). `-target=recovery` is the difference to rekeying a Shamir seal; without the flag OpenBao tries to rotate the barrier keys, and an auto seal does not have any.

I did **not** carry out this step here – this instance's keys have only seen one terminal. It is here as a procedure because it belongs in every runbook that describes a migration.

The key is not backed up

This is the point this guide cannot explain away. The token has no DKEK. `sc-hsm-tool --wrap-key` is therefore impossible; the seal key `openbao-unseal` exists exactly once. If the stick fails there is no replacement – and thus no way to the Raft data, not via the snapshot either, because that is encrypted with the same root key.

For the lab test that is acceptable. Before production use it is not, and the way there is clear:

1. Migrate back to Shamir (Part V, the way back) – while the stick is alive.
2. Re-initialise the token with `--dkek-shares 1`, create the share file, import it, store share and password separately.
3. Generate the key anew, export it (`--wrap-key`), store the export.
4. Migrate to PKCS#11 again. Procure a second stick, initialise it with the same share, import the key, restore drill with it.

That is an afternoon. It is cheaper than the day the stick stops blinking.

Snapshots remain mandatory

Auto-unseal changes nothing about N° 7. The snapshot CronJob keeps running; the restore drill has to know one more detail from now on: an instance that receives the snapshot needs the same seal – the same HSM, or the way back to Shamir before the snapshot. Playing a snapshot of a pkcs11-sealed OpenBao into a Shamir drill instance is rejected.

Checking that it runs

```
kubectl -n openbao get pod openbao-0 -o wide           # 1/1, NODE worker-03
kubectl -n openbao exec openbao-0 -- bao status        # Seal Type pkcs11, Sealed false
kubectl -n openbao logs openbao-0 | grep -i 'health'   # no warnings
ssh worker-03 'sudo journalctl -u pcsd --since -1h | grep -ci mismatch' # 0
ssh worker-03 'sc-hsm-tool | grep tries'              # User PIN tries left: 3
```

Part VIII – Debug: the network HSM does not answer (procedure)

This part is not a recorded session. It is the procedure I use to take an auto-unseal against an HSM on the network apart – Securosys Primus, Nitrokey NetHSM, Thales Luna, a cloud HSM with a PKCS#11 client. The commands are generic; the experience behind them comes from customer environments and from Parts III to VI of this guide.

The symptom is always the same: the pod starts, stays 0/1, and the log has a line with seal "pkcs11" and an error. The cause sits in one of seven layers, and the order in which you check them is not negotiable: **from the outside in, from dumb to clever**. Whoever starts with the OpenBao logs reads an error message that originated four layers further down.

The difference to the USB stick from Part II: between opensc-pkcs11.so and the chip there were pcscd and a socket. With a network HSM, that place is taken by the **vendor library** – a .so that speaks TCP, does TLS, reads its own configuration file and writes its own logs. It is the layer that most often stays silent.

The chain

Pod

```
bao seal "pkcs11" { lib, slot|token_label, key_label, ... }
| dlopen
| <vendor>-pkcs11.so -- reads --> /etc/<vendor>/*.cfg
| TCP + TLS (client certificate? password? token?)
```

NetworkPolicy · DNS · egress · firewall

▼
HSM appliance :<port> -- partition/slot -- key "openbao-unseal"
└─ its own logs: init, login, failed attempts

Step 1 – Can you get there at all?

From the pod, not from the node. The node has other routes, other policies and usually more rights:

```
kubectl -n openbao exec openbao-0 -- sh -c 'nc -zv -w 3 hsm.example.internal 2310'
kubectl -n openbao exec openbao-0 -- sh -c 'nslookup hsm.example.internal'
```

open or succeeded means layer 1 is done. timed out means firewall or NetworkPolicy; refused means the host answers but the port is closed – wrong port, wrong service, HSM not in operational state. can't resolve means DNS, and DNS from a pod is a different DNS than the node's (CoreDNS, ndots, search domains).

If the image has no nc: the debug pod from Part IV with the same image, or `kubectl debug -it openbao-0 --image=busybox --target=openbao`.

Step 2 – Is the library there, and does it load?

```
kubectrl -n openbao exec openbao-0 -- ls -la /usr/lib/<vendor>/
kubectrl -n openbao exec openbao-0 -- ldd /usr/lib/<vendor>/libprimusP11.so
```

ldd has to resolve every line. A not found is the error – and it is the error OpenBao reports as failed to load library or simply as CKR_GENERAL_ERROR, without saying which dependency is missing. Common: the library is built for glibc, the image is Alpine (musl) – the same trap as with the KMS plugin in Part X. Check with file <lib> – interpreter /lib64/ld-linux-x86-64.so.2 on Alpine is a no.

Second question in this layer: does the library find **its own configuration**? Every vendor has one – path, host, port, certificates, partition. Usually via an environment variable or a fixed path:

```
kubectrl -n openbao exec openbao-0 -- env | grep -iE 'PRIMUS|NETHSM|CHRYSTOKI|PKCS11'
kubectrl -n openbao exec openbao-0 -- cat /etc/primus/primus.cfg # or similar
```

If the file is missing in the pod because the ConfigMap mount was forgotten, the library sees no slot – and OpenBao says failed to find token. The same message as in Part III, a different cause.

Step 3 – Does the library see a slot?

```
pkcs11-tool --module /usr/lib/<vendor>/libprimusP11.so --list-slots
```

Expected: at least one slot with token label, token initialized. “No slots” means: the library runs but does not reach the HSM as a *client* – step 1 worked (TCP), but TLS fails, the client certificate has expired, the partition has a different name, or the library’s user does not exist on the appliance. From here on only the vendor log helps (step 7) – the PKCS#11 layer only says “no”.

Write down what it says here: **slot number and label exactly** – Part III showed that the label is what the library generates, not what the docs say.

Step 4 – Can the token do what OpenBao needs?

```
pkcs11-tool --module <lib> --list-mechanisms | grep -E 'AES-GCM|RSA-PKCS-0AEP'
```

OpenBao needs CKM_AES_GCM or CKM_RSA_PKCS_0AEP. If both are missing there is no configuration that works – only another key type or another HSM. If only AES-GCM is missing (NetHSM, SmartCard-HSM), the key is an RSA pair and mechanism = "CKM_RSA_PKCS_0AEP".

Step 5 – Login and key

```
pkcs11-tool --module <lib> --slot <n> --login -0
```

The PIN is prompted. Three outcomes:

- **CKR_PIN_INCORRECT** – wrong PIN. Do not guess again: network HSMs lock users after a few attempts, and a pod in CrashLoop guesses every minute. Check the Secret first (length, trailing newline – echo instead of printf is the classic), then once by hand.
- **Login ok, key missing** – key_label is wrong, or the key lies in a different partition, or the PIN's user is not allowed to see it (roles: operator vs. administrator).
- **Login ok, openbao-unseal present, with Usage: decrypt or unwrap** – layer 5 is done. If it only says sign, it is the wrong key or the wrong usage mask.

Step 6 – One operation, if possible

The PKCS#11 layer proves that the key is usable without OpenBao in between:

```
head -c 32 /dev/urandom > /tmp/plain
pkcs11-tool --module <lib> --slot <n> --login --encrypt \
  --id <key-id> -m RSA-PKCS-OAEP --hash-algorithm SHA256 -i /tmp/plain -o /tmp/enc
pkcs11-tool --module <lib> --slot <n> --login --decrypt \
  --id <key-id> -m RSA-PKCS-OAEP --hash-algorithm SHA256 -i /tmp/enc | cmp - /tmp/plain &&
echo OK
```

Not every library supports --encrypt via pkcs11-tool; then --sign with the same key is enough as proof that login and key access work. If only this step fails (CKR_MECHANISM_INVALID, CKR_KEY_FUNCTION_NOT_PERMITTED), it is the mechanism or the usage mask – not the network, not the PIN.

Step 7 – What the HSM itself says

Every appliance has a log, and it is better than anything the client side provides. Securosys Primus: the partition's audit/system log; NetHSM: nitropy nethsm logs or the syslog target; Luna: lunacm and the device's syslog. Look for the timestamp of the last pod start:

- *Connection from ..., TLS handshake failed* → certificate, cipher, time (a pod without NTP and an HSM with a strict clock do not get along)
- *Login failed for user ...* → PIN or user; read the failed-attempt counter
- *Session opened, key ... not found* → label/partition
- nothing at all → layer 1 or 2; the connection never arrives

Step 8 – Hold the OpenBao configuration against what you found

Only now the stanza – and the **rendered** one, not the one in the values:

```
kubectl -n openbao exec openbao-0 -- cat /openbao/config/extraconfig-from-values.hcl
kubectl -n openbao exec openbao-0 -- sh -c 'env | grep -E "^BA0_HSM_" | sed "s/=.*/=<set
>/'
```

Five values, five comparisons with steps 2 to 5: lib = the path ldd resolved cleanly; slot or token_label = exactly the output of step 3; key_label = exactly the output of step 5; mechanism = one from step 4; BA0_HSM_PIN set (length!) and not additionally pin in the stanza with a different value

– the environment variable wins, and guessing which one applies has already cost hours.

Step 9 – Mounts and Secrets

```
kubectl -n openbao describe pod openbao-0 | sed -n '/Mounts:/,/Conditions:/p'
kubectl -n openbao get secret openbao-hsm -o json | python3 -c "...len..." # Part IV
kubectl -n openbao get cm,secret -o name | grep -iE 'hsm|pkcs|primus'
```

What describe shows is the truth; what the values say is the intent. A volume missing in the StatefulSet because OnDelete never replaced the pod (Part IV) looks correct in the values.

Step 10 – Now the OpenBao logs

```
kubectl -n openbao logs openbao-0 --previous | grep -iE 'seal|pkcs|hsm|error'
```

--previous, because the current container in CrashLoop has not written anything yet. With the results from 1–9 the error message is now readable: failed to find token is layer 2/3, pin is required layer 9, CKR_MECHANISM_INVALID layer 4, this build of OpenBao has PKCS#11 disabled Part III.

Step 11 – From the node yes, from the pod no

If step 1 works from the node and not from the pod:

```
kubectl -n openbao get networkpolicy
kubectl -n openbao describe networkpolicy <name> # egress to the HSM port allowed?
kubectl -n openbao exec openbao-0 -- cat /etc/resolv.conf
```

A default-deny egress policy (Nº 7 has one for the drill instance) is the normal case in hardened namespaces. The rule has to allow the HSM port and DNS (53/UDP+TCP to CoreDNS). Service meshes with enforced mTLS are the second cause: a sidecar that wants to terminate TLS turns the vendor library's TLS handshake into an error in step 3.

Step 12 – When everything is right and it still does not work: strace

```
kubectl debug -it openbao-0 --image=alpine --target=openbao -- \
  sh -c 'apk add -q strace && strace -f -e trace=openat,connect -p 1 2>&1 | grep -iE "pkcs
|\.cfg|connect"'
```

Two questions only strace answers: does the process **really** open the library from lib (or another one, because a second path exists in the image)? And does the library open **its** configuration file – and which one? openat(... "/etc/primus/primus.cfg") = -1 ENOENT is in no OpenBao log. Neither is connect(... sin_port=htons(2310) ...) = -1 ECONNREFUSED.

The order, as a table

#	Layer	Command	Says on failure
1	network	nc -zv, nslookup from the pod	firewall, NetworkPolicy, DNS, port
2	library	ls, ldd, file, its .cfg	missing dependency, glibc/musl, missing mount
3	slot	--list-slots	TLS, certificate, partition, user
4	mechanism	--list-mechanisms	key type does not fit OpenBao
5	login/key	--login -0	PIN, lockout, label, role
6	operation	--encrypt/--decrypt or --sign	usage mask, mechanism
7	HSM log	appliance	TLS handshake, failed attempts, clock
8	stanza	rendered HCL + env	typo, env vs. stanza
9	pod	describe pod, Secret length	mount missing, Secret empty
10	OpenBao	logs --previous	now readable
11	policy	networkpolicy, resolv.conf, mesh	egress, DNS, mTLS
12	syscalls	strace -e openat,connect	which file, which port really

And the rule that holds all twelve together: **after every error found, start again at step 1**. Part VI showed why – every error hides the next, and the error message does not always change when the cause does.

Part IX – One HSM for all pods: the HSM VM (draft)

This part is not implemented. It describes what the next step would be and weighs the options.

The problem with the setup from Part IV: the stick hangs off one worker, so OpenBao hangs off one worker. An HA OpenBao with three replicas is not possible that way – three pods on three nodes would need three sticks with the same key (DKEK!) or an HSM that is reachable over the network. The goal: a dedicated VM holds the stick and makes it available to all OpenBao pods. Four ways, from the simplest to the proper one:

Way A – Transit seal via the VM OpenBao (recommended)

That was the original plan for N° 6, and after this guide it is more attractive than before. The OpenBao on the VM from N° 3 gets the stick and seal "pkcs11" – exactly as here, only as a systemd service instead of a pod, without protocol mismatch (client and daemon from the same Ubuntu), without nodeSelector, without hostPath. The cluster OpenBao then gets no PKCS#11 seal but a **transit seal**: it has its root key encrypted by the VM OpenBao via its transit engine.

```
Cluster pods (any number, any nodes)
  seal "transit" { address = "https://bao-vm:8200", key_name = "cluster-unseal", ... }
    | HTTPS, token with a policy only on transit/*/cluster-unseal
    ▼
VM OpenBao — seal "pkcs11" — pcsd — Nitrokey HSM 2
```

Advantages: no PKCS#11 in the cluster, no custom image, HA with three replicas possible immediately, the prepared files `cluster/transit-setup.sh` and `cluster/values-transit.yaml` fit. Disadvantage: a chain – the cluster hangs off the VM, the VM off the stick. If the VM fails, no cluster pod restarts any more; if it is **running**, it keeps running. The transit token needs rotation (periodic token, N° 3 has the pattern).

Way B – p11-kit remote: the PKCS#11 library over the network

p11-kit can export a PKCS#11 module over a Unix socket:

```
# on the HSM VM
p11-kit server --provider /usr/lib/x86_64-linux-gnu/opensc-pkcs11.so \
  "pkcs11:manufacturer=www.CardContact.de" -n /run/p11-kit/hsm.sock
```

In the pod, OpenBao loads the file `p11-kit-client.so` instead of `opensc-pkcs11.so` and gets `P11_KIT_SERVER_ADDRESS=unix:path=/run/p11-kit/hsm.sock` – where the socket in the pod comes from a sidecar that tunnels it to the VM (ssh -L, socat with TLS, or a WireGuard network). p11-kit itself has **no authentication and no encryption**; the PIN travels through this channel. The tunnel is mandatory, not optional.

Advantage: OpenBao keeps seal "pkcs11", the VM is a dumb PKCS#11 server without state of its own. Disadvantage: a sidecar per pod, a tunnel that has to be up before OpenBao, and p11-kit in the image – with the same pcsc-lite question on the VM side. Not tested.

Way C – USB/IP

The Linux kernel can export USB devices over IP (usbip). The HSM VM exports the stick, a worker imports it and sees it as a local device. That moves the stick from one worker to another but does not make it available to **all** pods at once – a USB device has one host. Usable for failover, not for HA. And USB/IP over an untrusted network is an unencrypted USB bus over an untrusted network.

Way D – Nitrokey NetHSM

Nitrokey's answer to exactly this question: an HSM with a REST API, its own PKCS#11 library (libnethsm_pkcs11.so, for glibc and musl), clustering, backup and an official OpenBao guide. There too RSA-OAEP applies (no AES-GCM), there too a custom image on an openbao-hsm base. There is a container test image for the rehearsal. The price is that of an appliance; for a lab it is the answer to "how do the big ones do it", not to "what do I build next week".

Recommendation

Way A. It uses what is there (VM OpenBao, Ansible role, prepared scripts), it brings HA for the cluster, and it keeps PKCS#11 in one place where it is simple. The stick moves from worker-03 to the VM (qm set), the cluster OpenBao migrates from pkcs11 to transit – an auto-seal-to-auto-seal migration for which OpenBao provides disabled = "true" on the old stanza. That will be N° 6b or an addendum to this guide.

Part X – OpenBao 2.7: from HSM build to KMS plugin (draft)

This part is not implemented either; 2.7.0 is in beta at the time of writing.

The first start of the HSM image had a warning in the log worth reading:

```
[WARN] The HSM distribution of OpenBao is discontinued and will no longer
receive updates beyond this minor version. PKCS#11 support has not been
removed, but is now available via an external KMS plugin that is drop-in
compatible with the previously built-in PKCS#11 seal.
```

openbao/openbao-hsm ends with 2.6.x. From 2.7 there is one standard image and a **KMS plugin mechanism**: auto seals that are no longer built in run as a separate process that OpenBao starts. The configuration gets one more stanza, the seal stanza stays:

```
plugin "kms" "pkcs11" {
  command = "openbao-plugin-kms-pkcs11"
  version = "v0.2.0"
}

seal "pkcs11" {
  lib      = "/usr/lib/pkcs11/opensc-pkcs11.so"
  token_label = "SmartCard-HSM"
  key_label  = "openbao-unseal"
  mechanism  = "CKM_RSA_PKCS_OAEP"
}
```

The name in seal must be the name from plugin. kms plugins can **only** be registered declaratively, not via the API – they have to be there before OpenBao is unsealed. The binary lies in plugin_directory; alternatively OpenBao pulls it as an OCI artifact (image = "...@sha256:...").

The stumbling blocks already visible

The plugin release (openbao-plugins, kms-pkcs11-v0.2.0, September 2026) ships openbao-plugin-kms-pkcs11_linux_amd64_v1 – a **glibc** binary (interpreter /lib64/ld-linux-x86-64.so.2). On the Alpine standard image (musl) it does not run without gcompat. The clean way is the UBI variant:

```
FROM openbao/openbao-ubi:2.7.x
USER root
RUN microdnf install -y opensc pcsc-lite-libs && microdnf clean all
COPY --chmod=0755 openbao-plugin-kms-pkcs11_linux_amd64_v1 /openbao/plugins/openbao-plugin-kms-pkcs11
USER openbao
```

Things to check about that as soon as 2.7.0 is out:

- **RHEL 9 ships pcsc-lite 1.9.x** – protocol 4:4. The pin from Part III would become unnecessary; the host's pcscd 2.0.3 fits.

- **OpenSC 0.23 in RHEL 9** – the token label will be SmartCard-HSM (UserPIN) again. Check after the rebuild, as always.
- `plugin_directory` in the server configuration has to point at the directory; the chart has `server.standalone.config` for that and – if needed – an `extraVolumes/volumes` mount for the plugin binaries.

The migration itself is meant to do without `unseal -migrate`: the plugin is “drop-in compatible” according to the release notes, the root key stays wrapped with the same HSM key. One would still test it in the kind context first – with a snapshot beforehand.

Appendix A – All commands

```
# — Proxmox host —————
lsusb | grep -i nitrokey
qm set <vmid> -usb0 host=20a0:4230
qm shutdown <vmid> && qm start <vmid>

# — Worker VM —————
sudo apt install -y pcscd libccid pcsc-tools opensc
sudo systemctl enable --now pcscd
sudo vi /etc/polkit-1/rules.d/40-allow-pcscd.rules          # see Part II
sudo systemctl restart polkit pcscd
pcsc_scan
sc-hsm-tool                                               # status, DKEK, PIN tries
sc-hsm-tool --initialize --dkek-shares 1 --label openbao # ONLY on a fresh stick
sc-hsm-tool --create-dkek-share dkek-share-1.pbe
sc-hsm-tool --import-dkek-share dkek-share-1.pbe
pkcs11-tool -M | grep -E 'AES|OAEP'                     # what the token can do
pkcs11-tool --login --keypairgen --key-type rsa:2048 --label openbao-unseal --id 10 --usage-decrypt
pkcs11-tool -0                                           # public part
sudo journalctl -u pcscd --since -15min                  # protocol mismatch?

# — Image —————
docker buildx build --platform linux/amd64,linux/arm64 -t softxpert/openbao-pkcs11:2.6.2 --push .
docker buildx imagetools inspect softxpert/openbao-pkcs11:2.6.2 # digest → values
trivy image --platform linux/amd64 --severity HIGH,CRITICAL softxpert/openbao-pkcs11:2.6.2

# — Cluster: preparation —————
kubectl apply -f pkcs11-debug.yaml                        # Part IV
kubectl -n openbao exec pkcs11-debug -- pkcs11-tool -L   # label from THIS image
read -rs "PIN?HSM user PIN: "; printf '%s' "$PIN" > /tmp/hsm-pin; unset PIN
kubectl -n openbao create secret generic openbao-hsm --from-file=BAO_HSM_PIN=/tmp/hsm-pin
rm -P /tmp/hsm-pin
kubectl -n openbao get secret openbao-hsm -o json | python3 -c "...len..." # 6–15 bytes!

# — Cluster: migration —————
kubectl -n openbao exec -it openbao-0 -- bao operator raft snapshot save /tmp/pre-pkcs11.snap
kubectl -n openbao cp openbao-0:/tmp/pre-pkcs11.snap ./pre-pkcs11.snap
helm upgrade openbao openbao/openbao --version 0.29.4 -n openbao -f helm/values.yaml
kubectl -n openbao delete pod openbao-0
kubectl -n openbao logs openbao-0 | grep -iE 'seal|error'
kubectl -n openbao exec -it openbao-0 -- bao operator unseal -migrate # x3, ONCE
kubectl -n openbao exec openbao-0 -- bao status

# — The proof —————
kubectl -n openbao delete pod openbao-0                 # enter nothing
kubectl -n openbao logs openbao-0 | grep unseal         # "unsealed with stored key"

# — Rotate the recovery keys (not carried out) —————
kubectl -n openbao exec -it openbao-0 -- bao operator rekey -target=recovery -init -key-shares=5 -key-threshold=3
kubectl -n openbao exec -it openbao-0 -- bao operator rekey -target=recovery -nonce=<nonce>
> # x3 OLD keys → 5 NEW

# — The way back (not carried out) —————
```

```
#   seal "pkcs11" { ... disabled = "true" } → helm upgrade → delete pod  
kubectl -n openbao exec -it openbao-0 -- bao operator unseal -migrate # x3 RECOVERY keys
```

Appendix B – Glossary

Seal / barrier – The barrier is OpenBao's encryption layer over the storage; the seal determines how the root key for it is protected: Shamir (enter keys) or auto seal (ask an external system).

Root key – Key with which OpenBao encrypts the storage. Lies encrypted in storage itself – with pkcs11, with an AES key that is RSA-OAEP-wrapped.

Recovery keys – The former unseal keys after a migration to auto seal. Authorise generate-root and the way back; cannot reconstruct the root key.

HSM – Hardware security module: generates, stores and uses keys without handing them out.

Nitrokey HSM 2 / SmartCard-HSM – USB stick with a SmartCard-HSM card from CardContact; OpenSC support, RSA/ECC, no AES-GCM.

PKCS#11 (Cryptoki) – Standard C API for HSMs and smartcards; a .so loaded via dlopen.

Slot / token / object – Socket / card in the socket / key or certificate on the card. Tokens have a label; the SmartCard-HSM's label is generated by OpenSC and changes between versions.

Mechanism – A cryptographic operation the token performs (CKM_RSA_PKCS_OAEP, CKM_AES_GCM). pkcs11-tool -M lists them.

User PIN / SO PIN – Login for key operations (3 tries) / administrative login for initialisation and PIN reset (15 tries).

DKEK – Device Key Encryption Key of the SmartCard-HSM. Only with a DKEK share can keys be exported encrypted and brought to a second stick. Set at --initialize.

OpenSC – Open-source implementation of PKCS#11 for smartcards; provides opensc-pkcs11.so, pkcs11-tool, sc-hsm-tool.

PC/SC, pcscd, libpcsc-lite – The reader abstraction below PKCS#11. pcscd is the daemon (speaks CCID/USB), libpcsc-lite the client library; both speak a versioned protocol over /run/pcscd/pcscd.comm. Client 4:5 against server 4:4 is rejected.

CCID – Chip Card Interface Device, the USB protocol for smartcard readers; libccid is the driver in pcscd.

Polkit – Authorisation service on Linux; pcscd asks it whether a process may use the reader.

HSM build (+hsm, cgo) – OpenBao binary with PKCS#11 support compiled in; as openbao/openbao-hsm up to 2.6.x, KMS plugin afterwards.

KMS plugin – From OpenBao 2.6/2.7: auto-seal mechanism as an external process, declared with plugin "kms" "<name>" in the server configuration.

-migrate – Flag of bao operator unseal that re-encrypts the root key from the old to the new seal. Once per migration, with the keys of the **old** seal.

OnDelete – StatefulSet update strategy: a changed template only takes effect when the pod is deleted. Chart default for OpenBao.

Digest pin – Image reference name: tag@sha256:...; the digest wins. Protects against overwritten tags and the node cache.

About the author

Thomas Zachmann is a freelance platform engineer based in Hamburg. He builds enterprise platforms for Kubernetes, cloud and AI workloads – from identity and secrets through CI/CD and GitOps to observability – so that the in-house team can run them without him afterwards. These Field Notes come out of that work. For project enquiries: thomaszachmann.de.

The pod could not find the HSM. The test pod next to it could.

This guide abolishes the unseal ritual from N° 1: afterwards the root key is encrypted with an RSA key that never leaves a Nitrokey HSM 2. The stick hangs off the Kubernetes worker, the pod gets the pcscd socket as a hostPath, a custom image brings OpenBao's HSM build and OpenSC. For readers without HSM experience, Part I explains what seal, token, SO PIN and DKEK mean – and why the last question comes before the first key.

The way there: the official image cannot do PKCS#11; pcsc-lite 2.4 in the container speaks 4:5, pcscd 2.0.3 on the host only 4:4; OpenSC 0.26 names the token differently than 0.23; a read -p in zsh yields an empty Secret. Then unseal -migrate, three times – and a pod that comes up without a key. Two drafts to close: one HSM for all pods, and the plugin path of OpenBao 2.7.

```
-- What the pod says, what the host says
Error configuring seal "pkcs11":
  failed to find token with label: SmartCard-HSM (UserPIN)

$ journalctl -u pcscd
Communication protocol mismatch!
Client protocol is 4:5 # pcsc-lite 2.4, Alpine 3.24
Server protocol is 4:4 # pcscd 2.0.3, Ubuntu 24.04
```

01	02	03	04	05	06
Terms	The stick	The image	Protocol 4:5	Migration	Drafts
seal, HSM, PKCS#11, DKEK	Proxmox, pcscd, Polkit, RSA key	openbao-hsm + OpenSC, multi-arch	pcscd rejects the client	unseal -migrate x3, the proof	HSM VM for all pods, OpenBao 2.7

openbao	2.6.2+hsm · chart 0.29.4 · standalone
hsm	Nitrokey HSM 2 · FW 4.1 · RSA-2048 · 0AEP
image	hsm-build · opensc 0.26 · pcsc-lite 2.2.3
host	Ubuntu 24.04 · pcscd 2.0.3 · polkit
seal	pkcs11 · recovery shamir 3/5

40 pages · 10 parts · 2 appendices
Deep dive: **Vault in Practice**
leanpub.com/vault-in-practice
Set in Inter Tight, Inter and JetBrains Mono.
All output is real terminal output.
CC BY 4.0 · share freely · no warranty